# FEE X-RAY

*Finding the money a small business is quietly losing to fees*

| | |
|---|---|
| Document | Fee X-ray, Product Requirements |
| Version | 2.0 |
| Status | Draft |
| Date | July 19, 2026 |
| Author | Satyam Pandey |
| Repository | github.com/SatyamPandey07/Fee-X-ray |
| License | Open source |

## In One Paragraph

Fee X-ray watches a small business's bank and payment processor accounts and tells the owner, in plain English, where they're bleeding money. It looks for overpriced card processing, subscriptions nobody uses anymore, bank fees that could have been waived with a phone call, and chargebacks that are about to expire unchallenged. It's not a budgeting app and it's not a bookkeeper. It's closer to an X-ray: something that looks past the surface of a bank statement and shows you what's actually there.

## A Concrete Example

Picture a small chain of coffee shops. The owner is busy running the business, not auditing statements. Meanwhile, three things are quietly happening: the payment processor is charging a rate slightly above the standard interchange rate, a project management tool the team stopped using eight months ago is still billing forty dollars a month, and a customer chargeback from six weeks ago has never been disputed and is two weeks from becoming permanent. None of this is hidden on purpose. It's just buried in normal transaction noise, and nobody has time to go looking for it.

> *That's the gap Fee X-ray is built to close: turning a pile of transactions nobody has time to read into three concrete, dollar-denominated things to go fix.*

## Who This Is Built For

- **The business owner.** Usually holds the Owner role. Connects the bank account, sees the findings, decides what to act on, and manages billing.
- **Someone doing the books.** May hold either role depending on how much trust the owner extends. Cares most about the findings dashboard and less about billing or connection management.
- **A read-only team member.** Holds the Member role. Can see findings and savings, but can't touch billing, connections, or organization settings.

## How the Pieces Fit Together

Fee X-ray isn't one application, it's three, and each one does a specific job. The Java core service is the system of record: it owns identity, organizations, roles, and billing state, and nothing about money or access changes without going through it. The Python analysis engine is where the actual financial reasoning happens: it pulls transactions and runs the fee detection rules against them. The Next.js frontend is simply the window into both of those, a dashboard rather than a place where business logic lives.

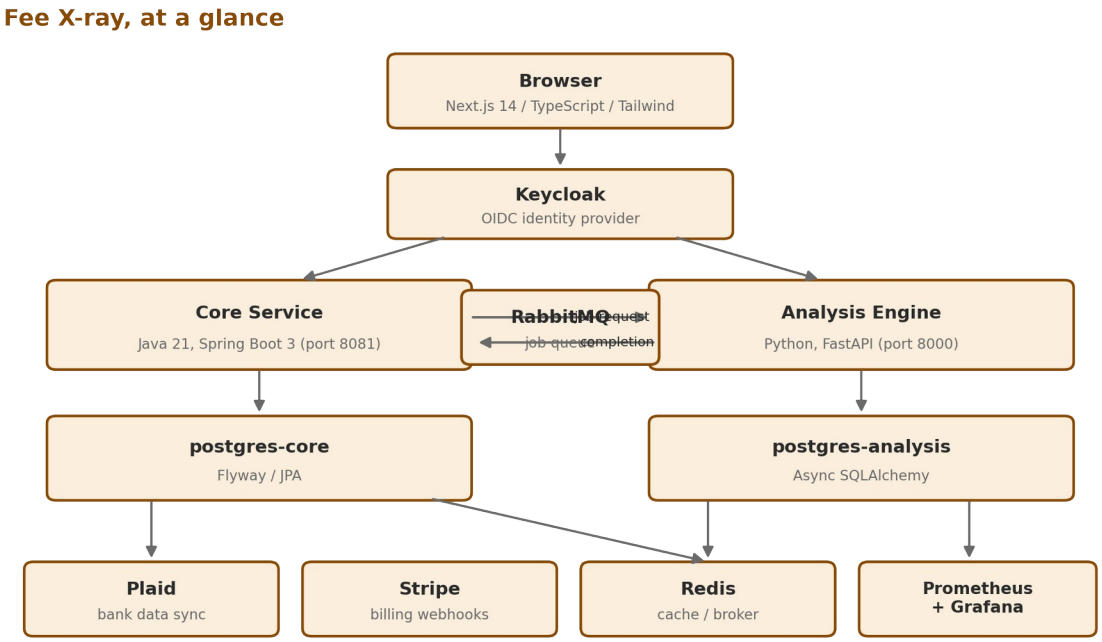


*Figure 1. The three services and the infrastructure around them.*

The split between Java and Python wasn't arbitrary. Identity, billing, and access rules need to be boringly correct every single time, which is where Spring Boot and a strict compiler earn their keep. Transaction analysis benefits more from fast iteration and a rich data ecosystem, which is why that half of the system is Python. Keeping them as separate services, talking only through RabbitMQ, means neither one can accidentally become tightly coupled to the other's internals.

## Walking Through a Real Analysis Run

It's easier to understand the system by following one request through it than by reading a component list. Here's what happens the moment a user clicks the button to run an analysis.

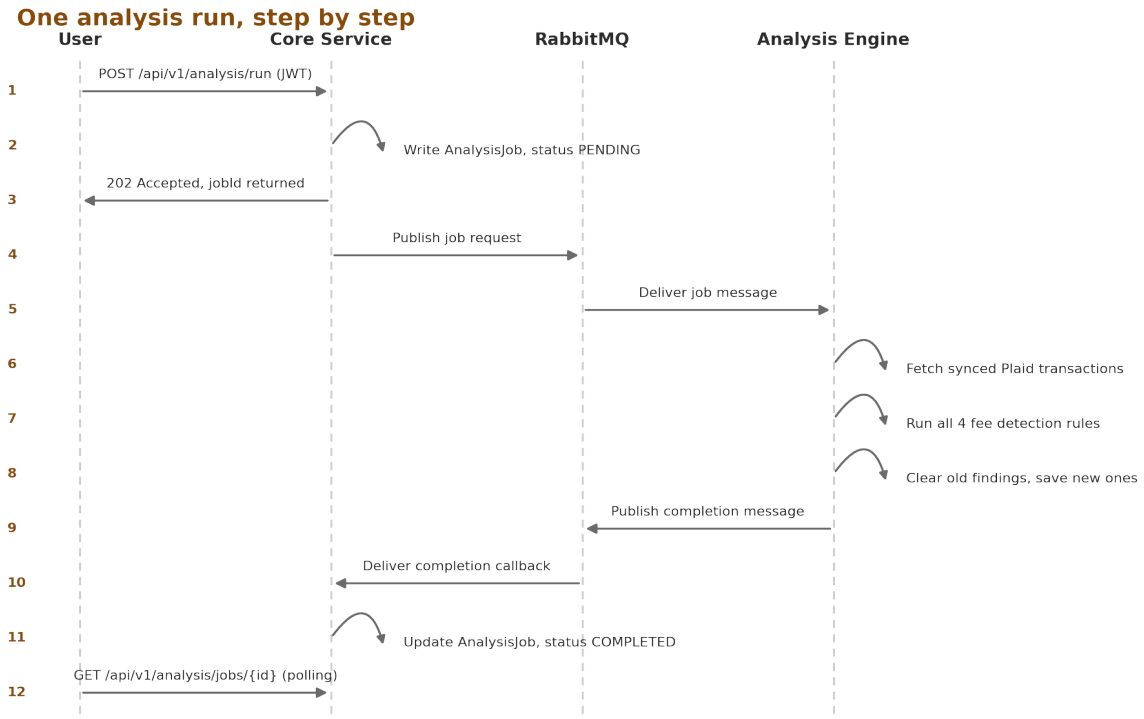


*Figure 2. A single analysis run, traced across all four participants.*

1. The user sends an authenticated request to the core service to start an analysis.
2. The core service immediately writes a job record marked pending and hands back a job ID. It doesn't wait around for the analysis to finish.
3. That job request goes onto a RabbitMQ queue rather than being called directly.
4. The analysis engine, listening on that queue, picks the job up whenever it's ready.
5. It fetches the organization's synced Plaid transactions and runs all four detection rules against them.
6. Old findings are cleared and the new ones are saved, so the dashboard never shows stale results next to fresh ones.
7. A completion message goes back over RabbitMQ.
8. The core service updates the job to completed, and the frontend, which has been polling quietly in the background, renders the results.

Notice what this buys the system: if the analysis engine is briefly slow or overloaded, the user still gets an immediate response and a job ID. The two services never block on each other synchronously.

## What It Actually Does

### Getting a business onboarded

| Capability | What happens |
|---|---|
| Login via Keycloak | OpenID Connect handles authentication; the frontend keeps the token in an httpOnly cookie and the core service checks it on every request. |
| First-login provisioning | The very first successful login for a new user creates their organization automatically and makes them the Owner. There's no separate signup |

| Capability | What happens |
|---|---|
| | form to fill out. |
| Bank connection via Plaid | Owners link real bank and payment processor accounts, and transactions begin syncing shortly after. |

### Finding the money

The rules engine is deliberately narrow rather than trying to catch everything. Four specific, well-understood problems, each one common enough in small business finances to be worth building a dedicated rule for.

| Rule | What it's looking for | Why it matters |
|---|---|---|
| Processor rate benchmarking | Card processing fees charged above standard interchange rates | Even a small percentage difference compounds across thousands of transactions a year |
| Zombie subscription detection | Recurring software charges with no user activity in the last 90 days | Easy to forget to cancel, easy money to get back |
| Unwaived bank fee detection | Overdraft, wire, and similar service charges | Many banks will waive these on request, but only if someone asks |
| Undisputed chargeback detection | Payment disputes with no matching reversal or win | These expire. Once the window closes, the money is gone for good |

### Staying visible after the first analysis

- Findings are explained in plain English rather than shown as raw transaction rows.
- A running savings total shows the cumulative value of findings the business has acted on, so the product keeps proving its worth well past the first login.
- Owners can manage settings and connections; members get a read-only view of the same findings and savings.

## The Data Model, Briefly

Three entities carry most of the weight in the core service's database, and the relationships between them are what tenant isolation is built on top of.

| Entity | Key Fields | Relationship |
|---|---|---|
| Organization | id, name, subscription_tier, created_at | Owns many users and many bank connections |
| User | id, org_id, email, keycloak_subject_id, role, created_at | Belongs to exactly one organization |
| Bank Connection | id, org_id, plaid_item_id, institution_name, status, connected_at | Belongs to exactly one organization |

Findings themselves aren't part of the published entity relationship diagram, but the behavior around them is well documented: each analysis run clears the previous set for an organization and writes a fresh one, so there's always exactly one current set of findings per organization rather than an ever-growing history to reconcile.

## The API Surface

The README documents two endpoints directly, both on the core service, and both tied to the analysis job lifecycle described above.

| Method | Path | Purpose |
|---|---|---|
| POST | /api/v1/analysis/run | Starts a new analysis job for the caller's organization. Requires a valid JWT. Returns 202 Accepted with a job ID. |
| GET | /api/v1/analysis/jobs/{jobId} | Returns the current status of a job, and once completed, a results summary. |

Endpoints for managing organizations, users, bank connections, and billing clearly exist as well, since the dashboard depends on them, but they aren't individually documented in the README. Worth listing as a gap to close rather than assuming their shape.

## Security and Tenant Isolation

This is a fintech product handling real banking data, so the security model isn't an afterthought bolted onto the architecture, it's the reason the architecture looks the way it does.

- Every login goes through Keycloak using OpenID Connect. The frontend never handles raw credentials directly.
- Tokens are stored in httpOnly cookies, out of reach of client-side scripts, and validated by the core service on every single API call.
- Every database query and modification is scoped to the caller's organization. There is no code path where one organization can see another's data.
- Plaid access tokens, which represent real access to a business's banking details, are encrypted with AES-128 before they ever touch the database, and the encryption keys themselves are rotated rather than fixed forever.

## Billing and Plans

| Plan | Bank Connections | Analysis Schedule | Price |
|---|---|---|---|
| Free | One active connection | Manual, on demand | No cost |
| Pro | Unlimited | Scheduled hourly, plus on demand | Flat monthly fee via Stripe |

Upgrades happen through a Stripe-hosted Checkout session rather than a custom payment form, and Stripe's webhook events are the source of truth for plan changes. When a webhook comes in, the organization's plan is updated in the core database in real time, so access and billing state never drift apart for long.

## Quality Gates and CI/CD

The repository's own GitHub Actions badges point to four separate pipelines, which says something about how seriously the project treats shipping safely, worth calling out on its own rather than folding into a generic testing paragraph.

| Pipeline | What it checks |
|---|---|
| CI | Runs the automated test suites for the core service, analysis engine, and frontend on every change |
| CD | Handles deployment once changes pass CI |
| CodeQL Analysis | Static analysis looking for security vulnerabilities and code quality issues |
| Secret Scan | Checks that no credentials, tokens, or keys are accidentally committed to the repository |

- Java core service tests run through Gradle: ./gradlew test
- Python analysis engine tests run through pytest
- Frontend checks run through npm run lint for linting and type checking

## What's Deliberately Out of Scope

- Taking remediation actions on the user's behalf, such as auto-cancelling a subscription or filing a chargeback dispute. Fee X-ray surfaces the finding; the human still makes the call and takes the action.
- Support for accounting platforms or ERPs. Connectivity is scoped to what Plaid covers for banks and payment processors.
- A dedicated mobile app. The frontend is a responsive web dashboard, not a native app.
- Continuous real-time transaction streaming. Analysis runs as a discrete job, not a live feed.

## Non-Functional Expectations

- The core service must remain the single source of truth for identity, organization, and billing state, no exceptions.
- Tenant isolation is not a soft guideline. Every query touching organization data must be scoped, and this should be enforceable through tests, not just code review.
- The two backend services stay decoupled through RabbitMQ. Neither should require a synchronous call to the other to function.
- Both backend services emit structured JSON logs with a correlated request ID, so a single request can be traced across service boundaries without guesswork.
- Prometheus metrics and the pre-loaded Grafana dashboard should give an operator a real answer about system health without needing to read logs first.

## What Success Looks Like

| Signal | What we'd want to see | How we'd check |
|---|---|---|
| Findings are accurate | Low false-positive rate across all | Manual review of flagged |

| Signal | What we'd want to see | How we'd check |
|---|---|---|
| | four rule categories | findings against known transaction histories |
| Value shows up fast | An organization sees at least one finding on its very first analysis run | Tracking time from first bank connection to first completed job |
| Savings keep growing | Tracked savings per organization trend upward as findings get acted on | Dashboard savings metric over time, aggregated across organizations |
| Free-to-Pro conversion | A meaningful share of organizations hitting the one-connection cap choose to upgrade | Stripe subscription events correlated with plan-limit hits |
| Jobs don't get stuck | Analysis jobs move cleanly from pending to completed | Grafana dashboards tracking job status transitions and queue depth |

## Assumptions We're Making

- Plaid's coverage of banks and payment processors is good enough for the target customer base. Fee X-ray inherits Plaid's limits as its own.
- Four rule categories are a meaningful starting point, not the final list. New fee patterns will surface as more organizations run analyses.
- Users are willing and able to act on findings themselves. The product's value depends on someone actually making that phone call or cancelling that subscription.
- Keycloak, RabbitMQ, and both Postgres databases are healthy most of the time. The system doesn't currently define what should happen if any of them go down mid-request.

## Where This Could Go Wrong

| Risk | Why it matters | What helps |
|---|---|---|
| A tenant isolation bug leaks data across organizations | This is the single worst thing that could happen to a fintech product handling banking data | Treat organization scoping as untestable-away, back it with dedicated tests, and lean on CodeQL to catch what review misses |
| A Plaid access token is exposed | Equivalent to leaking real banking access | AES-128 encryption at rest, dynamic key rotation, and secret scanning on every commit |
| A job gets stuck between RabbitMQ and the analysis engine | Leaves a user staring at a pending job with no explanation | Grafana visibility into job status transitions, with alerting on jobs that stall past an expected window |
| Findings feel noisy or wrong | Trust in the product collapses fast if the first finding someone checks turns out to be false | Keep each rule narrowly scoped, well-tested, and paired with a clear explanation of why it fired |
| A Stripe webhook is missed or delayed | An organization's access can drift out of sync with what they've | Treat webhook events as authoritative, but reconcile |

| Risk | Why it matters | What helps |
|---|---|---|
| | actually paid for | periodically against Stripe's own subscription records |

## Open Questions and Where We'd Go Next

These aren't formally scheduled work, just the natural next questions that follow from what's already built.

- What should happen to a job if the analysis engine or RabbitMQ is down when it's submitted? Right now this isn't documented.
- Should the rules engine grow beyond four categories, and if so, what's the process for adding a new one safely?
- Would a lightweight audit log of tenant-scoped data access be worth the overhead, given how central isolation is to the trust model here?
- The repository doesn't carry a specific license file today, though it's treated as open source; a clear license file would remove any ambiguity for anyone building on it.

## Quick Reference

| Item | Value |
|---|---|
| Core service | Java 21, Spring Boot 3, port 8081 |
| Analysis engine | Python, FastAPI, port 8000 |
| Frontend | Next.js 14, TypeScript, Tailwind CSS, port 3000 |
| Identity | Keycloak, port 8080, preconfigured realm and test users |
| Demo login | owner-demo / owner123 |
| Metrics | Prometheus on port 9090, Grafana on port 3001 (admin / admin) |
| Start everything | cd infra, then docker-compose up -d --build |
| Run core service tests | cd core-service, then ./gradlew test |
| Run analysis engine tests | cd analysis-engine, then pytest |
| Run frontend checks | cd frontend, then npm run lint |